



Linear Control Systems 25411

Instructor: Behzad Ahi

Assignment 1

Fall 1404

Due Date: 1404/9/1

Note: Bonus problems are indicated in *purple*.

Note: You are expected to upload a **PDF** file that includes a *detailed derivation* procedure for each question. For the simulation part, along with the essential plots, it is necessary to *append* your *code* at the end of the PDF file. Additionally, provide the *source codes* in a separate folder, where the name of each source code corresponds to the number of the related question.

1 Mathematical Model of a Motorcycle

Figure 1 illustrates a motorcycle with a rider. By leveraging arbitrary resources, address the following questions.

1.1 Develop a sequence of three mathematical models of the system in the vertical direction.

Hint: Consider the elasticity of the tires, the elasticity and damping of the struts (in the vertical direction), the masses of the wheels, and the elasticity, damping, and mass of the rider.

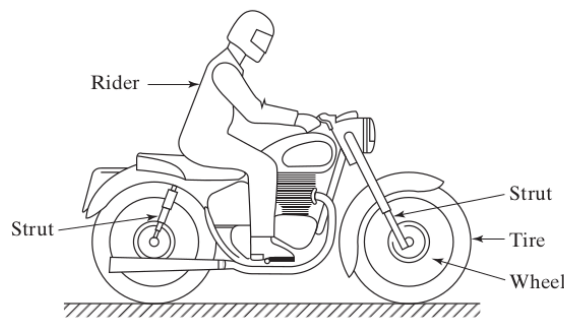


Figure 1: Motorcycle with a rider.

1.2 Derive the differential equation for each proposed mathematical model. Why these equations are important in control system design?

2 Human Eye Movement

A model for human eye movement consists of the closed-loop system shown in Figure 2, where an object's position is the input and the eye position is the output. The brain sends signals to the muscles that move the eye. These signals consist of the difference between the object's position and the position and rate information from the eye sent by the muscle spindles. The eye motion is modeled as an inertia and viscous damping and assumes no elasticity. Assuming that the delays in the brain and nervous system are negligible, answer the following questions.

2.1 find the closed-loop transfer function for the eye position control.

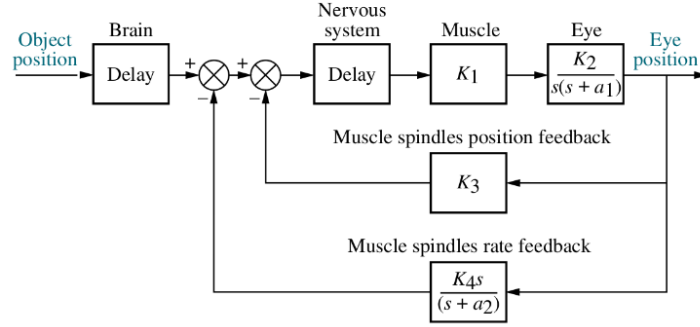


Figure 2: Feedback control system representing human eye movement.

2.2 Represent the system in state space, where the input is the object position and the output is the eye position. Use the observable form.

3 Hybrid Solar Cell and Diesel Power Distribution System

A model for Hybrid solar cell and diesel power distribution system proposed as shown in Figure 3, The system has been shown to have a very good uninterruptible power supply as well as line voltage regulation capabilities. The output V_{Load} , is the voltage across the load. The two inputs are I_{Cf} , the reference current, and I_{dist} , the disturbance representing current changes in the supply.

3.1 Referring to Figure 3, find $V_{Load}(s)/I_{Cf}(s)$ and $V_{Load}(s)/I_{dist}(s)$.

3.2 By choosing appropriate states, find the state-space representation in the form of the equations below, such that A , B , C , D , W , and V are matrices with appropriate dimensions.

$$\begin{aligned}\dot{X}(t) &= AX(t) + BI_{Cf}(t) + WI_{dist}(t) \\ V_{Load}(t) &= CX(t) + DI_{Cf}(t) + VI_{dist}(t)\end{aligned}$$

4 Forced Vibration with Coulomb Damping

A single-degree-of-freedom system with Coulomb (dry-friction) damping, subjected to a harmonic force, is shown in Figure 4.

4.1 By using the sign of $\dot{x}(t)$ to model Coulomb damping, derive the dynamic equation of motion.

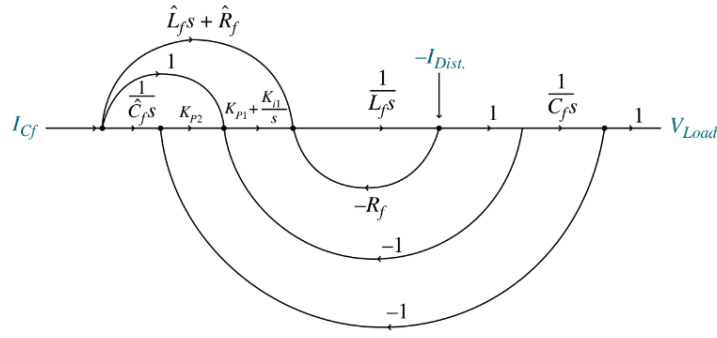


Figure 3: Hybrid solar cell and diesel power distribution system.

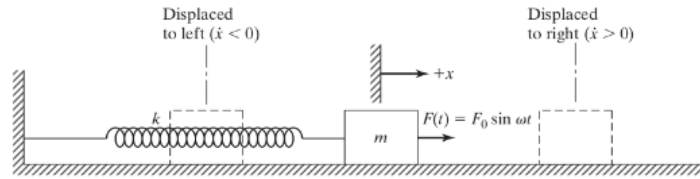


Figure 4: Single-degree-of-freedom system with Coulomb damping.

4.2 By choosing appropriate state variables, derive the nonlinear state-space representation.

5 Antenna Azimuth Position Control System

Consider the antenna azimuth position control system shown in Figure 5.

5.1 Search online to find the system's subsystems and annotate each in the Figure.

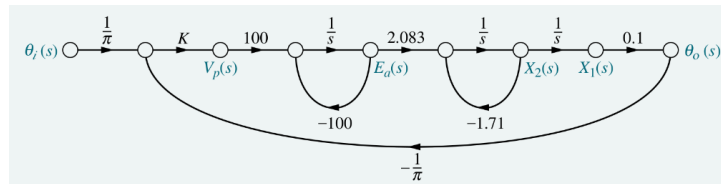


Figure 5: Signal-flow graph for the antenna azimuth position control system.

5.2 Use Mason's formula to find the closed-loop transfer function.

MATLAB & Simulink Assignments

Note: you just need to do **Assignments**. Read extra explanations if you're having trouble figuring out the exercise.

6 Response of Nonlinear State-Space Models Using ode45

To numerically find the response of a nonlinear system with state-space representation and an arbitrary input signal using Matlab, you can follow these steps

- (I) Define the state-space representation of the nonlinear system. This includes the state equations and output equation. Let's assume that the system has three states, $x_1(t)$, $x_2(t)$, and $x_3(t)$, and the input is $u(t)$.

The state equations can be written as

$$\begin{aligned}\frac{dx_1(t)}{dt} &= f_1(x_1(t), x_2(t), x_3(t), u(t)) \\ \frac{dx_2(t)}{dt} &= f_2(x_1(t), x_2(t), x_3(t), u(t)) \\ \frac{dx_3(t)}{dt} &= f_3(x_1(t), x_2(t), x_3(t), u(t))\end{aligned}$$

The output equation can be written as

$$y(t) = g(x_1(t), x_2(t), x_3(t), u(t))$$

- (II) Define the time span and initial conditions. You need to specify the time span over which you want to simulate the system and the initial conditions of the states. Let's assume that you want to simulate the system from $t = 0$ to $t = 10$ seconds with an initial condition of $x_1(0) = 0$, $x_2(0) = 0$, and $x_3(0) = 0$.

```
tspan = [0 10];
x0 = [0; 0; 0];
```

- (III) Define the input signal. You need to specify the input signal $u(t)$ that you want to apply to the system. This can be a step function, a sinusoidal function, or any other arbitrary function.
- (IV) Define the functions for the state equations and output equation. You can define function handles for the state equations and input equation using anonymous functions in Matlab. For example, for the following system:

$$\begin{cases} \dot{x}_1(t) = -x_1(t)x_2(t) + \sin(x_3(t)) \\ \dot{x}_2(t) = 3e^{-x_1(t)} - \sqrt{x_3(t)} \\ \dot{x}_3(t) = -x_3^3(t) + \frac{u(t)}{x_1(t) + x_2(t)} \\ y(t) = x_1^2(t) - x_3(t) \end{cases}$$

```
function dxdt = f(t, x)
    u = input(t); % define input separately
    dxdt = [-x(1)*x(2) + sin(x(3));
            3*exp(-x(1)) - sqrt(x(3));
            -x(3)^3 + u/(x(1) + x(2))];
end
```

```
function y = g(x, u)
    y = x(1)^2 - x(3);
end
```

- (V) Call the `ode45` solver to integrate the system:

```
[t, X] = ode45(@f, tspan, x0);
for i = 1:length(t)
    u(i) = input(t(i));
    Y(i) = g(X(i,:), u(i));
end
```

Assignment:

Consider a *nonlinear spring-mass-damper system* governed by the following equation of motion:

$$m\ddot{x}(t) + c\dot{x}(t) + k_1x(t) + k_3x^3(t) = F_0 \sin(\omega t)$$

where

- $m = 10$ kg is the mass,
- $c = 1.2$ N·s/m is the damping coefficient,
- $k_1 = 4000$ N/m is the linear stiffness coefficient,
- $k_3 = 1 \times 10^5$ N/m³ is the cubic nonlinear stiffness coefficient,
- $F_0 = 100$ N is the amplitude of the external harmonic force,
- ω is the excitation frequency in rad/s.

Define the state variables as

$$x_1(t) = x(t), \quad x_2(t) = \dot{x}(t)$$

so that the system can be written in the nonlinear state-space form:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -\frac{c}{m}x_2 - \frac{k_1}{m}x_1 - \frac{k_3}{m}x_1^3 + \frac{F_0}{m} \sin(\omega t) \end{cases}$$

Simulate the system using the `ode45` function in MATLAB over the time interval $t \in [0, 10]$ seconds, with the initial conditions

$$x_1(0) = 0, \quad x_2(0) = 0.$$

Repeat the simulation for $\omega = 1$ Hz, $\omega = 10$ Hz, and $\omega = 100$ Hz, and examine the influence of excitation frequency on the amplitude and character of the nonlinear vibration response.

7 Simulation of a Nonlinear System in Simulink

To simulate the response of a nonlinear system in SIMULINK, consider a Continuous Stirred Tank Reactor (CSTR) with a first-order, exothermic, irreversible reaction $A \rightarrow B$. The reactor is well-mixed, with reactants continuously fed and products continuously removed. The dynamic model is given by:

$$\begin{cases} \frac{dC_A}{dt} = \frac{F}{V}(C_{A0} - C_A) - k_0 e^{-E/(RT)} C_A \\ \frac{dT}{dt} = \frac{F}{V}(T_0 - T) + \frac{-\Delta H}{\rho C_p} k_0 e^{-E/(RT)} C_A + \frac{Q}{\rho C_p V} \end{cases}$$

where:

- C_A = concentration of A in the reactor (mol/L)
- T = reactor temperature (K)
- F = volumetric flow rate (L/min)
- V = reactor volume (L)
- C_{A0}, T_0 = inlet concentration and temperature
- k_0, E = pre-exponential factor and activation energy
- R = gas constant

- ΔH = heat of reaction (J/mol)
- ρ, C_p = density and heat capacity of the mixture
- Q = heat removal/addition rate (J/min)

(I) Define the system parameters and equations. The reaction rate constant is:

$$k(T) = k_0 e^{-E/(RT)}$$

The state vector is $x(t) = [C_A(t), T(t)]^\top$.

(II) Create a MATLAB Function block for the dynamics. In SIMULINK, insert a **MATLAB Function** block and define the system as:

```
function dx = cstrDynamics(t, x)
    % States: x(1) = C_A, x(2) = T
    F = 100; V = 100; C_A0 = 1.0; T0 = 350;
    k0 = 7.2e10; E = 8750; R = 1.987;
    dH = -5e4; rho = 1000; Cp = 0.239;
    Q = -2e5; % Cooling rate
    k = k0 * exp(-E/(R*x(2)));
    dx = zeros(2,1);
    dx(1) = (F/V)*(C_A0 - x(1)) - k*x(1);
    dx(2) = (F/V)*(T0 - x(2)) ...
            + (-dH/(rho*Cp))*k*x(1) + Q/(rho*Cp*V);
end
```

- (III) Add Integrator blocks. Use two **Integrator** blocks to represent C_A and T . Set their initial conditions, e.g., $C_A(0) = 0.5$ mol/L, $T(0) = 350$ K.
- (IV) Connect the blocks. Feed the current states into the MATLAB Function block, and send the derivatives back to the integrators.
- (V) Run the simulation and observe results. Use **Scope** blocks to plot $C_A(t)$ and $T(t)$. Observe transient and steady-state behavior.

Assignment: Duplicate the results obtained in the previous Assignment in SIMULINK. Make sure the obtained results are similar.

8 Methods to Find the Response of a Linear System

In this section, we focus on using the `lsim` function in MATLAB to simulate the time response of a linear time-invariant (LTI) system to arbitrary inputs and initial conditions. The `lsim` function works with LTI objects such as transfer functions, state-space models, or zero-pole-gain models.

- (I) Define your system as an LTI object. You can use built-in functions such as `tf`, `ss`, or `zpk` to create these objects. For example, for the transfer function

$$T(s) = \frac{1}{s+1}$$

you can write:

```
sys = tf(1, [1, 1]);
```

(II) Combine multiple subsystems if needed. MATLAB provides commands to interconnect LTI systems:

- `series(sys1, sys2)` — connects two systems in series.
- `parallel(sys1, sys2)` — connects two systems in parallel.
- `feedback(sys1, sys2)` — forms a feedback connection between two systems.

For example, to connect two transfer functions in series:

```
sys_series = series(sys1, sys2);
```

(III) Specify the input signal and time vector. The input signal `u` should be the same length as the time vector `t`. For example, for a unit step input over 10 seconds with sampling time `Ts`:

```
t = 0:Ts:10;  
u = ones(size(t));
```

(IV) (Optional) Specify initial conditions. If your system is in state-space form, you can pass an initial state vector `x0`.

(V) Simulate the response using `lsim`. Pass the system object, input signal, and time vector (and optionally initial conditions) to `lsim`:

```
y = lsim(sys, u, t);  
% or with initial conditions:  
y = lsim(sys, u, t, x0);
```

Assignment: Using the proposed commands in MATLAB, create the system shown in Figure 5. Simulate the antenna azimuth position control system for 10 seconds with different inputs such as step and sine. Use both the transfer function and state-space models, and show that the results are the same.

9 Simulating the Response of a Linear System in Simulink

In this section, we focus on using **SIMULINK** to simulate the time response of a linear time-invariant (LTI) system to arbitrary inputs and initial conditions. We will demonstrate the procedure using the **Transfer Function** block and the **State-Space** block from the Simulink library.

(I) Open Simulink and create a new model. In MATLAB, type:

```
simulink
```

This opens the Simulink Library Browser and allows you to create a new blank model.

(II) Insert the system representation.

- For a transfer function $T(s) = \frac{1}{s+1}$, drag a **Transfer Function** block from **Simulink** → **Continuous** into your model and set:

$$\text{Numerator} = [1], \quad \text{Denominator} = [1 \quad 1]$$

- For a state-space model defined by matrices A, B, C, D , drag a **State-Space** block from **Simulink** → **Continuous** and enter:

$$A = [-1], \quad B = [1], \quad C = [1], \quad D = [0]$$

(III) Add an input source. From **Simulink** → **Sources**, drag a block such as:

- **Step** — for a step input.

- **Sine Wave** — for sinusoidal input.
 - **Signal Builder** or **From Workspace** — for custom signals.
- (IV) Add an output sink. From **Simulink** → **Sinks**, drag a **Scope** block to visualize the output, or use a **To Workspace** block to export simulation data to MATLAB.
- (V) Run the simulation. Set the desired stop time (e.g., 10 seconds) in the toolbar, choose an appropriate solver if needed (e.g., **ode45**), then click the **Run** button. View the output in the Scope or in MATLAB if exported.

Assignment: Duplicate the results obtained for the antenna azimuth position control system in the previous Assignment in SIMULINK. Make sure that results are similar.